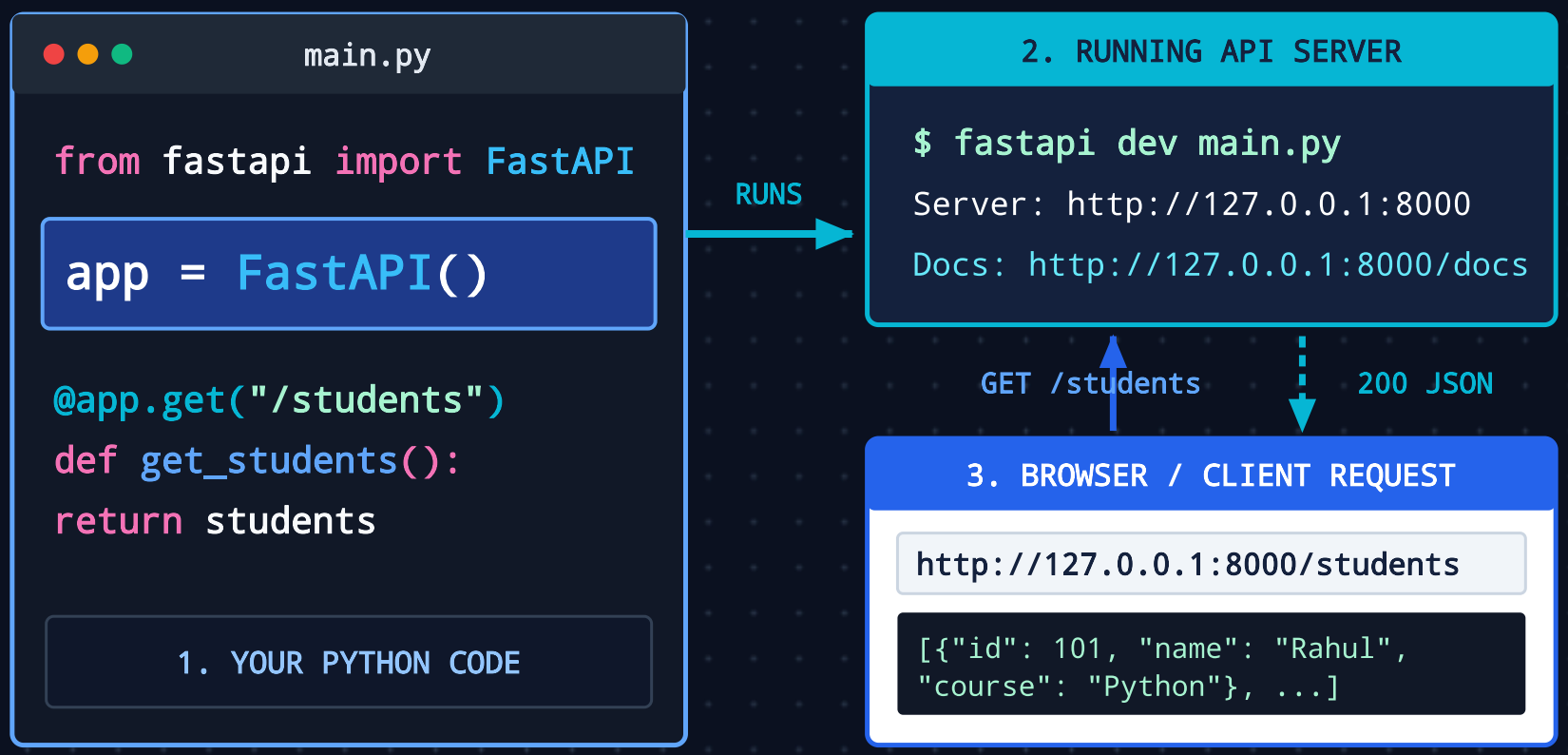
FROM THEORY TO WORKING PYTHON CODE

BUILD YOUR FIRST API WITH FASTAPI

> “From writing Python code to creating your first working endpoint.”

FIG 06.1 // PYTHON SOURCE CODE TO LIVE API TRANSFORMATION main.py → 127.0.0.1:8000



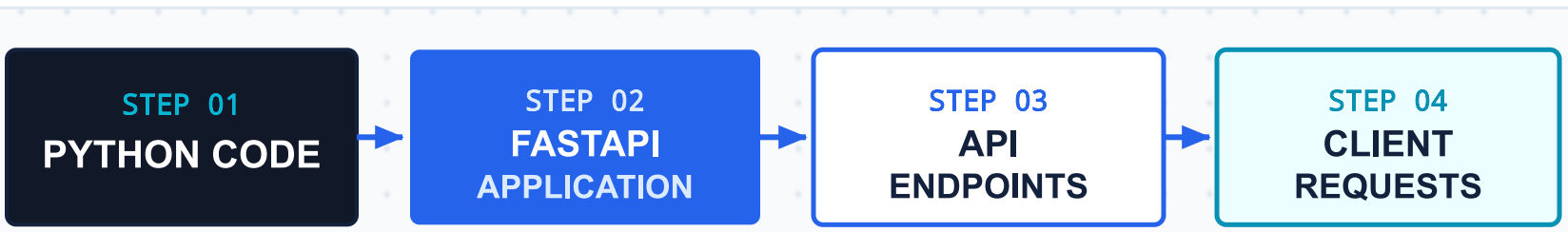
HANDS-ON PROJECT	ENDPOINTS BUILT	LIVE TESTING
Student Management API	3 Working GET Routes	Swagger UI (/docs)

BRIDGING PYTHON & THE WEB

Meet FastAPI. Your Python tool for building APIs.

FastAPI is a modern Python web framework for building APIs. It turns ordinary Python functions into web endpoints that clients can call.

FIG 06.2 // HOW FASTAPI CONNECTS PYTHON CODE TO CLIENT REQUESTS 4-STAGE PIPELINE



WHAT FASTAPI HELPS DEVELOPERS DO:

01. Define API endpoints
02. Receive HTTP requests
03. Validate input data
04. Execute Python functions
05. Return structured JSON responses automatically

BUILT AROUND MODERN PYTHON

FastAPI uses standard **Python type hints** (such as `int` and `str`) to support automatic data validation and interactive API documentation.

ISOLATING PROJECT DEPENDENCIES

Before writing code, prepare your environment.

A **virtual environment** keeps your project's installed packages separate from other Python projects on your computer.

PROJECT DIRECTORY

```
student_api/
│
├── .venv/
│
└── main.py
```

Note: `.venv` is a common convention, though you can name the environment folder anything.

TERMINAL — student_api SHELL

```
# 1. Create a virtual environment
$ python -m venv .venv

# 2A. Activate on Windows
> .venv\Scripts\activate

# 2B. Activate on macOS / Linux
$ source .venv/bin/activate

# 3. Install FastAPI + standard dependencies
(.venv) $ pip install "fastapi[standard]"
```

STEP 01 // CREATE

Creates an isolated Python environment folder—it does not install FastAPI yet.

STEP 02 // ACTIVATE

Switches your terminal session into `.venv` (command depends on your OS).

STEP 03 // INSTALL

Installs FastAPI along with `uvicorn` and the `fastapi` CLI tool.

INITIALIZING THE APPLICATION

Let's write our first FastAPI application.

Open `main.py` and write these two foundational lines of Python code:

student_api/main.py — PYTHON 3.10+

```python
from fastapi import FastAPI
app = FastAPI()
```

LINE 01

LINE 02

01 // THE IMPORT

`from fastapi import FastAPI`

Imports the `FastAPI` Python class from the installed package so we can build an application.

02 // THE INSTANCE

`app = FastAPI()`

Creates an **instance** of the FastAPI application and stores it in the variable `app`.

WHAT DOES "INSTANCE" MEAN IN PLAIN ENGLISH?

Think of `FastAPI()` as constructing your new API application, and `app` as the variable name you use to attach routes to it.

DECORATORS & ROUTE FUNCTIONS

Let's create an endpoint.

FIG 06.3 // ANATOMY OF A FASTAPI ROUTE & HANDLER FUNCTION

GET / → home()



`@app.get("/")`	**Route Decorator:** Tells FastAPI to handle `GET` requests sent to the root path `/`.
`def home():`	**Route Function:** The Python function FastAPI executes whenever that route is requested.
`return {...}`	**Return Value:** Returns a Python dictionary, which FastAPI automatically converts into JSON.

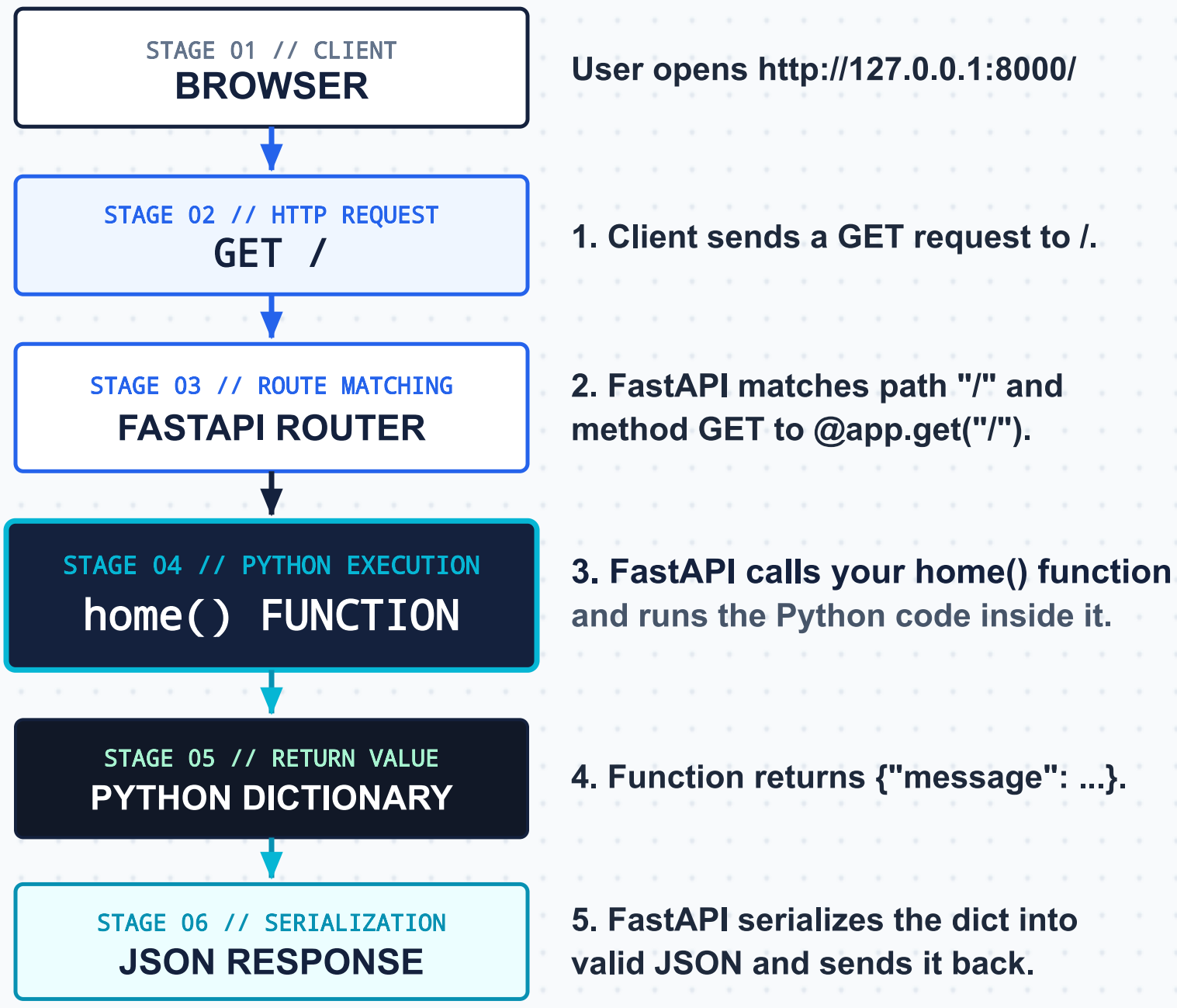
Highlight: An endpoint connects an HTTP request to a Python function.

BEHIND THE SCENES

What happens when someone visits your endpoint?

FIG 06.4 // END-TO-END FASTAPI REQUEST PROCESSING FLOW

6 EXECUTION STAGES



You write standard Python dicts—FastAPI handles HTTP & JSON serialization.

STRUCTURING IN-MEMORY DATA

Let's make the API more useful.

Add a Python **list of dictionaries** to store three student records in `main.py`:

student_api/main.py — Sample Student Dataset LIST[DICT]

```
students = [
    {"id": 101, "name": "Rahul", "course": "Python"},
    {"id": 102, "name": "Priya", "course": "Data Science"},
    {"id": 103, "name": "Aman", "course": "Machine
Learning"}
]
```

LIST INDEX	"ID" (INT)	"NAME" (STR)	"COURSE" (STR)
students[0]	101	Rahul	Python
students[1]	102	Priya	Data Science
students[2]	103	Aman	Machine Learning

WHY A LIST OF DICTIONARIES?

Each dictionary stores one student's fields, and the list lets us iterate through multiple records and convert directly to a JSON array.

IN-MEMORY STORAGE NOTE

This list lives in temporary memory (RAM), not a permanent database—any changes reset when the server restarts.

RETURNING MULTIPLE RECORDS

Let's retrieve all student records.

student_api/main.py — Collection Endpoint GET /students

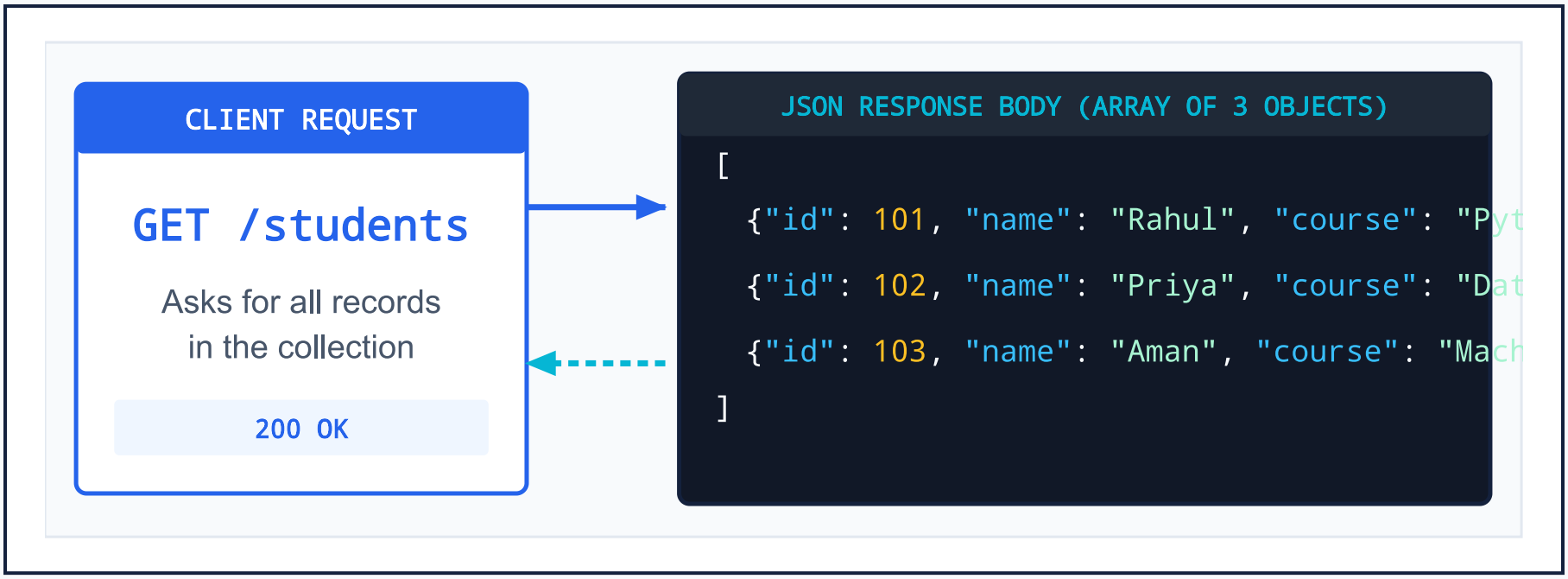
```python
@app.get("/students")
def get_students():
    return students
```

`@app.get("/students")`
Registers a `GET` endpoint at the path `/students`.

`def get_students():`
The route handler function FastAPI executes on request.

`return students`
Returns the Python list, which becomes a JSON array.

FIG 06.5 // REQUESTING THE COMPLETE STUDENT LIST — PYTHON LIST → JSON ARRAY



SEARCHING BY PATH PARAMETER

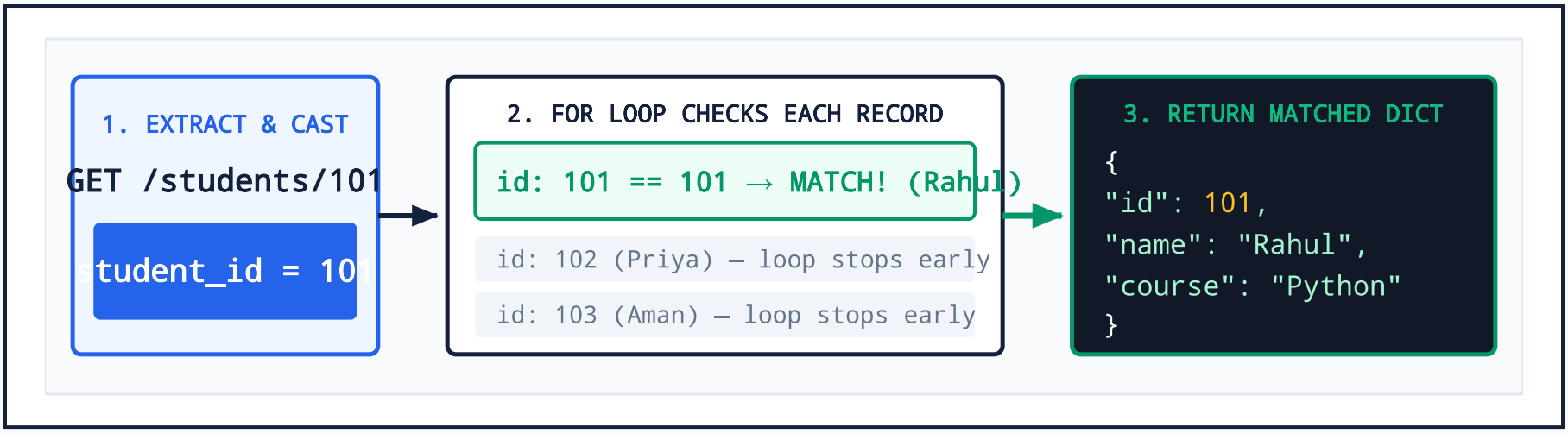
What if we only want one student?

student_api/main.py — Single Student Lookup GET /students/{student_id}

```python
@app.get("/students/{student_id}")
def get_student(student_id: int):
    for student in students:
        if student["id"] == student_id:
            return student

    return {"message": "Student not found"}
```

FIG 06.6 // LOOP EXECUTION FOR: GET /students/101 (student_id = 101) MATCH FOUND AT ID 101



IMPORTANT PRODUCTION NOTE

Returning a plain dictionary (`{"message": "Student not found"}`) still sends a `200 OK` status code. In a later part, we will use FastAPI's `HTTPException` to return a proper `404 Not Found` error.

LAUNCHING THE DEVELOPMENT SERVER

Your API is ready. Let's run it.

RECOMMENDED CLI COMMAND — FASTAPI CLI

```
$ fastapi dev main.py
```

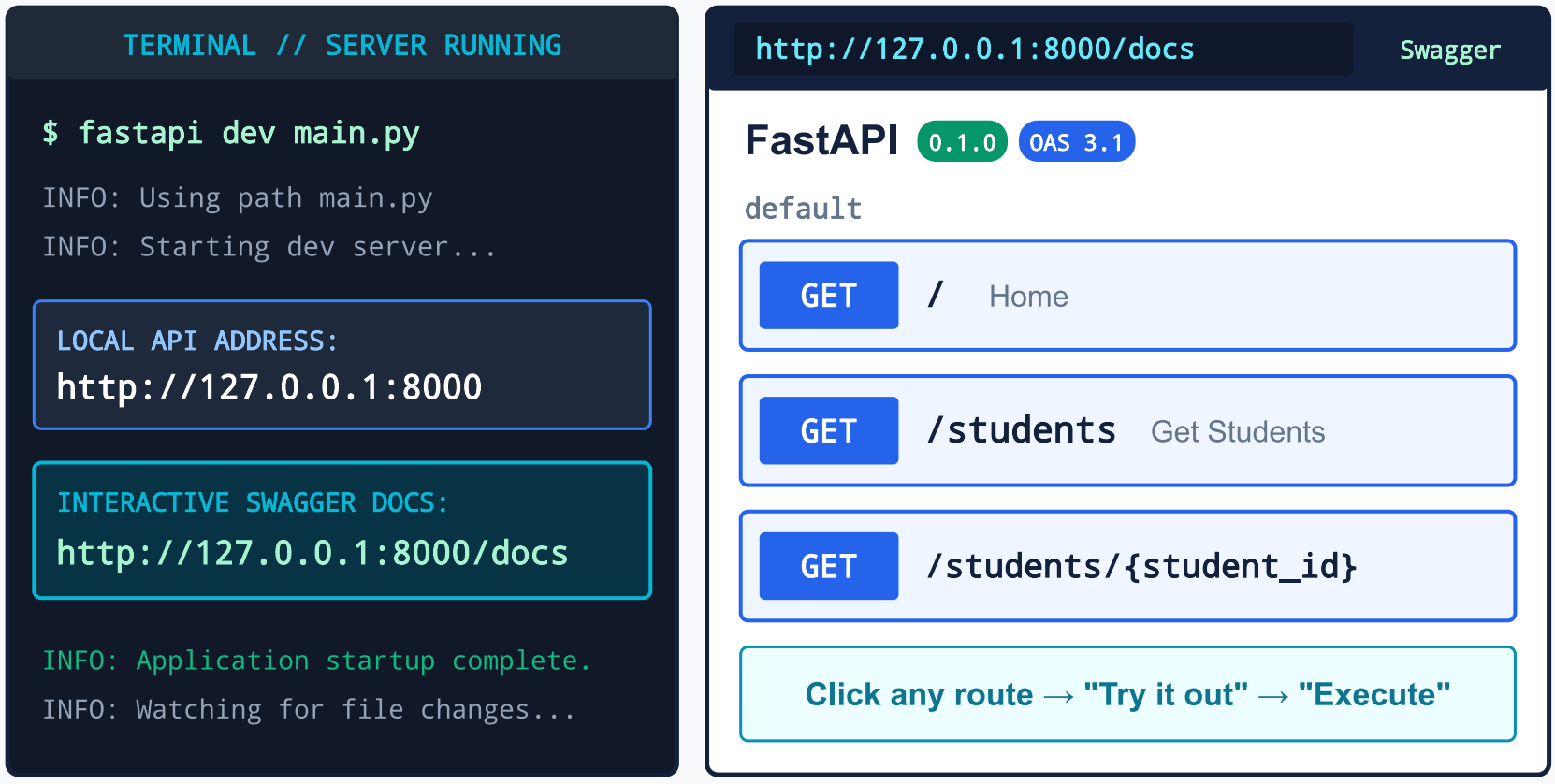
Starts the local development server with auto-reload enabled.

ALTERNATIVE UVICORN COMMAND — ASGI SERVER

```
$ uvicorn main:app --reload
```

`main` = file `main.py` • `app` = `FastAPI()` instance • `--reload` = auto-restart.

FIG 06.7 // LIVE SERVER TERMINAL & AUTOMATIC SWAGGER UI (/docs) http://127.0.0.1:8000/docs



YOUR FIRST WORKING PYTHON BACKEND

You just built your first FastAPI application.

student_api/main.py — Complete Working Application 22 LINES OF PYTHON